

Sistemas de Mensagens ({Messaging Systems

O que você aprenderá nesta página

Criar uma arquitetura complexa de microsserviços usando NATS como sistema de mensagens.

As filas de mensagens (*Message queues*) são um método de comunicação entre serviços. Elas têm uma ampla variedade de casos de uso e são úteis quando se deseja escalar aplicações. Diversos serviços baseados em APIs REST HTTP que desejam se comunicar entre si exigem que um conheça o endereço do outro. Em contrapartida, ao usar filas de mensagens, as mensagens são enviadas e recebidas a partir da própria fila.

Nesta seção, usaremos um sistema de mensagens chamado **NATS** para explorar os benefícios da mensageria. Antes de começarmos, vamos revisar os conceitos básicos do NATS.

No NATS, as aplicações se comunicam enviando e recebendo mensagens. Estas mensagens são endereçadas e identificadas por assuntos (*subjects*). O emissor *publica* a mensagem com um assunto. Os receptores *assinam* os assuntos para receber as mensagens publicadas. No modelo padrão publicação-assinatura (*publish-subscribe*), todos os assinantes do assunto recebem a mensagem publicada. Também é possível usar um modelo de fila (*queue*), no qual cada mensagem publicada é entregue a apenas **um** assinante.

O NATS oferece diferentes semânticas de entrega de mensagens ou modos de operação. A funcionalidade básica fornecida pelo *Core NATS* é o envio da mensagem *no máximo uma vez*: se não houver assinantes ouvindo o assunto (nenhuma correspondência de assunto) ou se não estiverem ativos no momento em que a mensagem é enviada, a mensagem não será recebida. Utilizando a funcionalidade *Jetstream*, também é possível alcançar a entrega de mensagens *pelo menos uma vez* ou *exatamente uma vez*, com persistência.

Com isso em mente, podemos projetar nossa primeira aplicação que usa mensageria para se comunicar.

Temos um conjunto de dados de 100.000 objetos JSON que exigem um processamento pesado, e depois precisamos salvar os dados processados. Infelizmente, processar um único objeto JSON leva tanto tempo que processar todos os dados exigiria horas de trabalho. Para resolver isso, dividi a aplicação em serviços menores que podem ser escalados individualmente.

A aplicação está dividida em 3 partes:

Fetcher: busca dados não processados e os repassa ao NATS. **Mapper**: processa os dados do NATS e, após o processamento, os envia de volta ao NATS. **Saver**: recebe os dados processados do NATS e, finalmente, (poderia) salvá-los.

Como já mencionado, a troca de mensagens no NATS é baseada em *assuntos*. Em geral, existe um assunto para cada finalidade. A aplicação utiliza quatro assuntos:

O **Fetcher** divide os dados em blocos (*chunks*) de 100 objetos e mantém um registro de quais blocos ainda não foram processados. A aplicação foi projetada de modo que o **Fetcher** não possa ser escalado.

O **Fetcher** se inscreve no assunto `mapper_status` e aguarda até que um **Mapper** publique uma mensagem confirmando que está pronto para processar dados. Quando o **Fetcher** recebe esta

informação, ele publica um bloco de dados no assunto `mapper_data` e recomeça o ciclo.

Como mencionado, quando um mapper está pronto para processar mais dados, ele publica essa informação de disponibilidade no assunto `mapper_status`. Ele também assina o assunto `mapper_data`. Quando o Mapper recebe uma mensagem, ele a processa, publica os dados processados no assunto `saver_data` e começa tudo novamente. O assunto `mapper_data` opera em modo de fila, então cada mensagem publicada é recebida por apenas um Mapper.

O **Saver** assina o assunto `saver_data`. Ao receber uma mensagem, ele a salva e publica uma mensagem de confirmação (*acknowledgement*) no assunto `processed_data`. O Fetcher assina esse assunto e acompanha quais blocos de dados já foram salvos. Assim, mesmo que alguma parte da aplicação falhe, todos os dados acabarão sendo processados e salvos. Além disso, o assunto `saver_data` também é utilizado no modo de fila, garantindo que cada bloco de dados processados seja gerido por apenas um Saver.

Para simplificar, o salvamento no banco de dados e a busca na API externa foram omitidos em nossa aplicação.

Para obter uma descrição mais detalhada da aplicação, consulte o pseudocódigo ou o código-fonte nas referências.

Antes de implantar o aplicativo, utilizaremos o Helm chart para instalar o NATS no nosso cluster:

```
helm repo add nats https://nats-io.github.io/k8s/helm/charts/
helm repo update
helm upgrade --install my-nats nats/nats \
  --namespace nats \
  --create-namespace \
  --set promExporter.enabled=true
```

Agora podemos executar a aplicação. Pelos logs, é possível perceber que os dados são transmitidos para o saver:

```
$ kubectl logs saver-dep-7548c99df-lhjsl
Received package 473. And data of length: 100
Received package 747. And data of length: 100
Received package 184. And data of length: 100
Received package 463. And data of length: 100
Received package 224. And data of length: 100
Received package 237. And data of length: 100
Received package 582. And data of length: 100
Received package 53. And data of length: 100
Received package 492. And data of length: 100
Received package 478. And data of length: 100
Received package 142. And data of length: 100
```

Além do NATS, também instalamos o NATS Prometheus exporter, que expõe métricas do servidor NATS em formato Prometheus no endpoint `/metrics`. O Prometheus pode então rastrear (*scrape*) essas métricas e podemos usar o Grafana para visualizá-las em *dashboards* para monitoramento em tempo real.

Vamos agora habilitar o monitoramento. Primeiro iniciamos o Prometheus e o Grafana de forma semelhante ao Capítulo 3.

O próximo passo é informar ao Prometheus onde encontrar o endpoint de métricas do pod NATS. Para isso, aplicamos o seguinte arquivo de valor `nats-prom-values.yaml`:

```
alertmanager:
  enabled: false

pushgateway:
  enabled: false
```

```
server:
  remotewrite: []

# Configuracao de rastreamento (scrape) estatica para metricas do NATS
extraScrapeConfigs: |
  - job_name: nats
    metrics_path: /metrics
    static_configs:
      - targets:
        - my-nats-0.my-nats-headless.nats.svc.cluster.local:7777
```

O comando é o seguinte:

```
helm upgrade --install prom prometheus-community/prometheus \
  --namespace monitoring \
  --create-namespace \
  --values nats-prom-values.yaml
```

Em termos mais técnicos, essa configuração adiciona uma tarefa de rastreamento (*scrape job*) do Prometheus, chamada *nats*, direcionada a `my-nats-0.my-nats-headless.nats.svc.cluster.local:7777`, e rastreia periodicamente o endpoint `/metrics` para a ingestão de métricas.

Como de praxe, podemos utilizar o Grafana para visualizar os dados. Em vez de escrever as consultas manualmente, usamos uma configuração já pronta aplicando `grafana-values.yaml`, que cria um painel (*dashboard*) com quatro quadros (*panels*). Aplicamos o arquivo de valores com o comando:

```
helm upgrade --install grafana grafana/grafana \
  --namespace monitoring \
  --values grafana-values.yaml
```

Agora estamos prontos para abrir o Grafana. Como de costume, fazemos o redirecionamento de porta (*port-forward*):

```
kubectrl port-forward -n monitoring svc/grafana 3000:80
```

O painel será exibido e as consultas PromQL correspondentes aos gráficos são as seguintes:

```
NATS target up/down {{(up{job="nats"})}} Connection count (sum(nats_varz_connections))
Message throughput in/out (rate(nats_varz_in_msgs[5m]),
rate(nats_varz_out_msgs[5m])) Byte throughput in/out
(rate(nats_varz_in_bytes[5m]), rate(nats_varz_out_bytes[5m]))
```

Crie um novo serviço separado para enviar mensagens de status das tarefas (*todos*) para *algum* serviço de chat. Vamos chamar o novo serviço de "broadcaster".

Requisitos:

O backend deve enviar uma mensagem para o NATS ao salvar ou atualizar as tarefas. O *broadcaster* deve assinar as mensagens do NATS. O *broadcaster* deve repassar a mensagem para **um serviço externo** em um formato que ele suporte.

Para o serviço externo, você pode escolher um destes: Discord, Telegram ou Slack. Ou, se não quiser usá-los, utilize a opção "Generic", na qual uma URL é definida como variável de ambiente e o *payload* é, por exemplo:

```
{
  "user": "bot",
  "message": "A todo was created"
}
```

O *broadcaster* deve ter capacidade de ser escalado sem enviar a mensagem várias vezes. Teste se ele funciona bem rodando com 6 réplicas. As mensagens só precisam ser enviadas para o serviço externo se todos os serviços estiverem operando corretamente. Portanto, uma perda aleatória de mensagem não é problema, mas o envio duplicado é.